

# Cross-references

Give a chunk a label and you can refer to its output from the prose with Typst's `@label` syntax. Use a recognized prefix so *Calepin* knows what the label points at; a `fig-` label attaches to figure output.

There are three ways to attach a label. Use exactly one of them per chunk.

## label argument

The clearest place for a label is the `label` argument of `#calepin.chunk`, alongside the caption:

In prose we mention Figure 1.

```
plot(mpg ~ hp, data = mtcars)
```

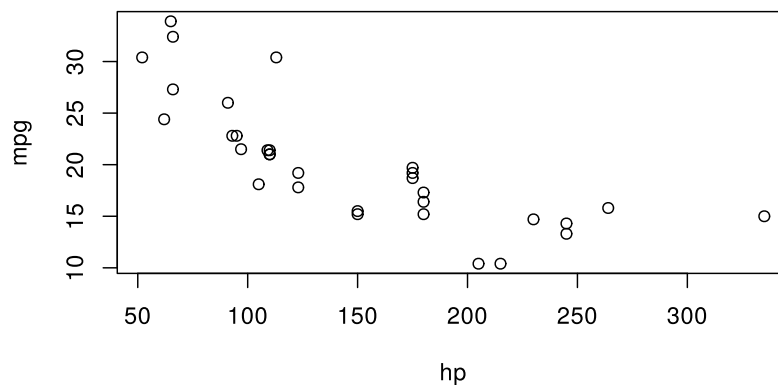


Figure 1: Miles per gallon and horsepower

## #| header

Put `#| label:` at the top of a plain fenced block when you want the label next to other chunk options:

```
```r
#| label: fig-cross-qmd
#| fig-caption: Distribution of car weights
#| fig-alt-text: Histogram of car weights
hist(mtcars$wt, col = "gray80", border = "white")
```
```

In prose we mention Figure 2.

```
hist(mtcars$wt, col = "gray80", border = "white")
```

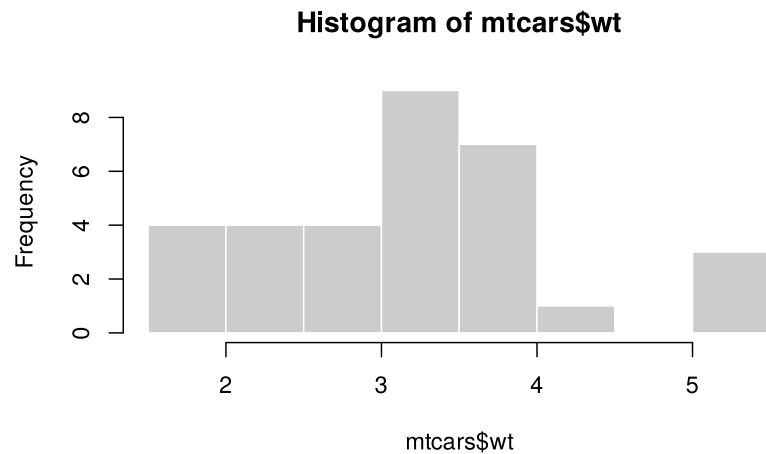


Figure 2: Distribution of car weights

## Trailing fence label

For a plain fenced block, you can also write a single label right after the closing fence. This is the most compact form, equivalent to one `#| label: header`:

```
```r
plot(dist ~ speed, data = cars)
```<fig-cross-trailing>
```
```

In prose we mention Figure 3.

```
```r
plot(dist ~ speed, data = cars)
```<fig-cross-trailing>
```
```

```
plot(dist ~ speed, data = cars)
```

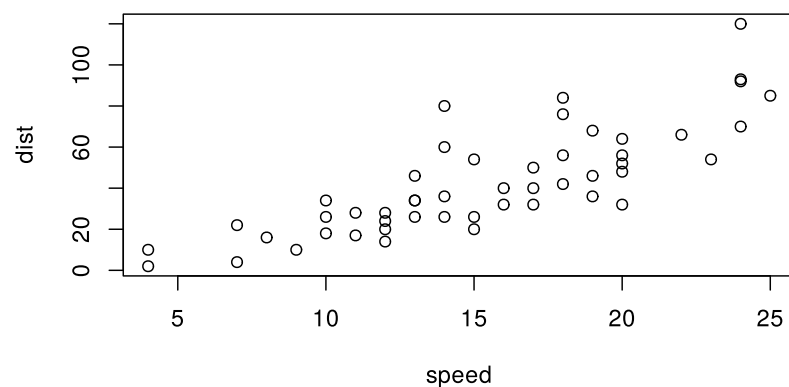


Figure 3: Speed and stopping distance

## Label prefixes

Use a recognized prefix so *Calepin* knows where the label belongs. A label without a recognized prefix, such as `label: "myplot"`, is still a valid chunk identifier (you can look it up with `#calepin.results`), but it is not a cross-reference, so `@myplot` will not resolve.

| Prefix | Target                       | Caption option |
|--------|------------------------------|----------------|
| fig-   | Figure or plot output        | fig-caption    |
| tbl-   | The chunk's non-image output | tbl-caption    |
| lst-   | The chunk's echoed source    | lst-caption    |

Each kind numbers from its own Typst counter, so a document can hold Figure 1, Table 1, and Listing 1 at once.

## Tables

A `tbl-` label wraps everything the chunk printed, so it works whatever produced the table — `knitr::kable`, a plain `print()` of a data frame, or text you assembled yourself.

In prose we mention Table 1.

|                   | mpg  | cyl | disp |
|-------------------|------|-----|------|
| Mazda RX4         | 21.0 | 6   | 160  |
| Mazda RX4 Wag     | 21.0 | 6   | 160  |
| Datsun 710        | 22.8 | 4   | 108  |
| Hornet 4 Drive    | 21.4 | 6   | 258  |
| Hornet Sportabout | 18.7 | 8   | 360  |
| Valiant           | 18.1 | 6   | 225  |

Table 1: First rows of `mtcars`

## Listings

An `lst-` label names the code itself rather than what it produced, so the chunk must echo its source (`echo: true`, the default).

In prose we mention Listing 1.

```
fit <- lm(mpg ~ hp, data = mtcars)
```

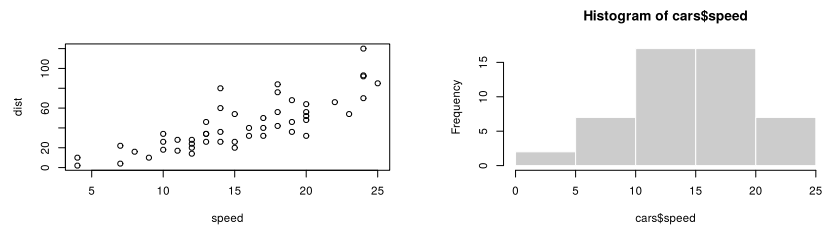
Listing 1: Fitting the model

A chunk can carry one label per kind, so `label: ("fig-plot", "lst-plot")` names both the plot and the code that drew it.

## Panels

A chunk that draws several plots lays them out in a grid. Give it `fig-subcaptions` and each panel becomes a sub-figure, lettered within its parent and referenceable on its own: `@fig-name-1` is the first panel, `@fig-name-2` the second, and a reference reads “Figure 1b”.

In prose we mention Figure 4, Figure 4a and Figure 4b.



(a) Scatter

(b) Histogram

Figure 4: Speed and stopping distance

A grid without sub-captions or a fig- label is left alone: its panels get no letters and consume no numbers.